

Solution Requirements

Date	2 July 2026
Team ID	
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

- **Functional Requirements (FR)** define specific behaviors, functions, or features of the system (What the system should *do*).
- **Non-Functional Requirements (NFR)** specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should *be*).

Fill out the descriptions and necessary details for each category provided below.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	Basic session-based validation for researcher dashboard login.	Medium
2	Authorization levels	Administrator can manage dataset variables; Farmer/Researcher has access to recommendations, compatibility checks, and dashboards.	Medium
3	External interfaces	Connects to local stats database (crop_stats.json) and Chart.js libraries.	High

		Exposes endpoints for crop data visualizers.	High
4	Transactions processing	Validates NPK, pH, and climate parameters to output recommended crop name via Random Forest Model.	High
5	Reporting	Renders interactive dashboard graphs of average nutrient levels, weather envelopes, and pH requirements.	High
6	Business rules, etc.	Enforces agricultural validation rules (pH between 0-14, humidity <= 100%, nutrients > 0).	High
7	Compliance to laws or regulations	Adheres to local digital data privacy guidelines for agriculture and secure web standards.	Medium
8	<i>Other</i>	No other extra requirements.	Low

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria

1	Performance & Speed	The ML model must return crop recommendation outputs in less than 1 second. Web page response time must be under 1.5 seconds.	Model response time <= 1.0s
2	Scalability	The Flask application structure must support concurrent queries from multiple users without performance degradation.	Supports 50+ concurrent requests
3	Security & Data Privacy	Input parameters must be sanitized on both frontend and backend to prevent script injection.	100% input sanitization check
4	Reliability & Availability	The Flask local server must run reliably during diagnostic farming sessions.	99.9% uptime in local tests
5	Usability & Accessibility	Interactive dashboards must be mobile-friendly and accessible across common web browsers.	Responsive viewport scaling checked
6	Other	Clean modular Python code with descriptive documentation for future development.	100% code documentation coverage